

CSED312 Project 3

Final Report

팀 오우노

20230355 정지성

20230345 이성재

목차

1. Frame Table
2. Lazy Loading
3. Supplemental Page Table
4. Stack Growth
5. File Memory Mapping
6. Swap Table
7. On Process Termination

1. Frame Table

1.1. 목표

Frame Table은 가상 메모리 관리의 핵심 구성 요소로, 여러 사용자 프로세스에 할당된 모든 물리 메모리 프레임을 효과적으로 관리하는 것을 목표로 한다.

현재의 Pintos 구현에서는 페이지 테이블 관리가 `userprog/pagedir.c`에 구현되어 있고, 페이지 테이블의 할당/해제는 `threads/palloc.c`에 구현되어 있다. 하지만 각 프레임에 대한 추가적인 정보를 관리하는 구조가 없어, 메모리가 부족할 때 적절한 프레임을 선택하여 evict 하는 등의 고급 메모리 관리 기능을 구현할 수 없다.

따라서 이 부분의 구현 목표는 다음과 같다.

1. 각 프레임의 정보를 효과적으로 관리하는 frame table 생성
2. 프레임 할당/해제 기능 구현
3. Free frame이 없을 때 victim을 선택하여 프레임을 반환하는 기능 구현
4. 사용자 프로세스(thread)가 사용하는 프레임을 검색하는 기능 구현

1.2. 구현방식

1.2.1. 자료구조

```
struct frame
{
    void *kaddr;           // kernel address, or physical address mapped to kernel space
    void *uaddr;           // user virtual address
    struct thread *owner;   // process that owns this frame.

    bool pinned;           // should we evict?

    struct list_elem elem;  // for the global frame_table list
};
```

위는 각 프레임에 대한 정보를 저장하는 구조체이다. (src/vm/frame.h:8-18) 필드는 다음과 같다.

- `kaddr`: 프레임의 커널 가상 주소로, frame number의 역할을 한다.
- `uaddr`: 이 프레임에 매핑된 사용자 가상 주소이다.
- `owner`: 이 프레임을 소유한 thread로, thread id의 역할을 한다.
- `pinned`: 프레임이 evict 되기로 결정된 경우, 다른 프로세스가 해당 프레임을 evict 하려고 하면 안될 것이다. 따라서 evict 과정이 진행중에는 true로 설정하고 다른 프로세스가 victim을 정하기 전에 pinned가 true인지 아닌지 확인하도록 했다.
- `elem`: frame_table 리스트에 속하기 위해 존재하는 list element이다.

frame.c에 선언된 전역 변수들은 다음과 같다.

- `struct list frame_table`: 할당된 모든 프레임을 관리하는 전역 리스트이다. Frame은 물리 메모리의 영역을 말하고 이는 모든 프로세스가 공유하기 때문에 전역 변수로 선언했다.
- `struct lock frame_table_lock`: Frame에 접근하는 것을 관리하는 lock이다.
- `static struct list_elem *clock_hand`: Clock 알고리즘 구현을 위한 포인터이다.

1.2.2. 알고리즘

1.2.2.1. 프레임 할당 알고리즘 (src/vm/frame.c:36-77)

프레임 할당은 다음과 같은 절차로 이루어진다.

1. 동기화를 위해 `frame_table_lock`을 획득한다.
2. 커널 메모리에서 새로운 `struct frame` 구조체를 할당한다.
3. `pallocc_get_page(PAL_USER)`를 사용하여 user pool로부터 프레임 획득을 시도한다.
 - 프레임은 반드시 user pool을 사용하여 할당되어야 한다.
4. 할당이 실패하면, Eviction 알고리즘을 호출한다.
5. frame 메타데이터를 초기화한다.
6. 전역 `frame_table`에 frame을 추가한다.
7. lock을 해제하고 frame 포인터를 반환한다.

이 알고리즘은 물리 메모리를 효율적으로 할당하면서, 메모리가 부족할 때 자동으로 페이지 교체를 수행한다.

1.2.2.2. Eviction: Clock 알고리즘 (src/vm/frame.c:122-161)

Free frame이 없을 때 victim을 선택하는 알고리즘으로 clock algorithm을 구현하였다. 이는 LRU의 근사 알고리즘으로, 평균 $O(1)$ 의 복잡도를 가진다.

1. `clock_hand`가 null이면 `frame_table`의 시작으로 초기화한다.
2. `clock_hand`를 순환 포인터로 사용하여 프레임들을 반복 순회한다.
3. Pinned==true인 프레임은 다른 프로세스에서 evict 시키고 있는 프레임이므로 건너뛴다.
4. Unpinned 프레임의 경우, `pagedir_is_accessed()`를 사용하여 하드웨어의 access bit을 확인한다.
5. 최근 접근되었다면(access bit = 1), access bit를 지우고 second chance를 부여한다.
6. 최근 접근되지 않았다면(access bit = 0) evict 대상으로 선택한다:
 - 동시 접근 방지를 위해 프레임을 pinned로 표시한다.
 - Spt를 참조해 페이지의 타입에 따라 swap 공간 또는 파일 시스템으로 데이터를 전송한다.
 - 현재 매핑에서 프레임을 제거한다.

1.2.2.3. 프레임 해제 알고리즘 (src/vm/frame.c:82-97)

프레임 해제는 다음과 같이 이루어진다.

1. `frame_table_lock`을 획득한다.
2. `pallocc_free_page()`를 사용하여 물리 페이지를 해제한다.
3. 해제되는 프레임을 `clock_hand`가 가리키고 있다면 다음 프레임으로 업데이트한다.
4. `frame_table` 리스트에서 프레임을 제거한다.
5. `struct frame` 메모리를 free한다.
6. lock을 해제한다.

1.3. 논의

1.3.1. Design report와 달라진 점

최초 디자인에서는 단순한 프레임 관리만을 생각했으나, 구현 과정에서 다음과 같은 기능들이 추가로 필요했다.

1. Pinning 메커니즘: Eviction 중인 프레임을 다른 프로세스가 victim으로 선택하는 경우를 방지하기 위해 pinned 플래그를 추가했다.
2. Clock hand의 업데이트: 프레임 해제 시 clock hand가 해제되는 프레임을 가리키고 있을 경우를 처리하는 로직이 필요했다(다음 프레임을 가리키도록).

1.3.2. 배운점

페이지 교체 알고리즘을 직접 구현하면서 OS의 메모리 관리 정책이 실제로 어떻게 동작하는지 이해할 수 있었다. Clock 알고리즘이 LRU의 좋은 근사치를 제공하면서도 구현이 간단하다는 점이 인상적이었다.

1.3.3. 한계점

Executable같이 출처가 file이었던 frame의 경우, dirty bit이 설정되지 않았다면 굳이 swap에 넣지 않고 정보를 버린 후, spte의 상태를 `FILE_LAZY`로 바꾸는 것이 swap의 공간 management 관점에서 훨씬 효율적인 구현일 것이다. 우리는 mmap으로 할당되지 않은 모든 frame에 대해, evict 대상으로 선정된 경우 추가적인 검사를 거치지 않고 바로 swap에 넣는다. 이를 해결하기 위해서는 `spt_entry`에 `bool file_backed`같은 플래그를 추가한 뒤 evict 직전에 이 플래그를 검사하는 방식으로 해결 가능할 것으로 예상된다.

2. Lazy Loading

2.1. 목표

기존의 Pintos는 유저 프로그램을 실행할 때 모든 코드를 물리적 메모리에 할당한다. 그러나 이렇게 하면 메모리 공간의 낭비가 심하기 때문에 lazy loading을 구현한다. 또한, lazy loading을 구현하면 우리가 가지고 있는 물리 메모리보다 더욱 커다란 파일을 여러 개 열어도 OS의 효율적 스케줄링으로 실행을 시키는 것이 가능하다.

2.2. 구현 방식

Loading이 필요할 때마다 `spt_initialize_file_as_lazy()` 함수를 불러와 사용했다. 해당 함수의 기능은 다음과 같다.

- 기존 Pintos의 `load_segment()`와 유사한 parameter을 argument로 받는다.
- 받은 parameter을 토대로, 현재의 process가 가지는 supplemental page table에서 `PAGE_LAZY` 상태를 갖는 S-page table entry(spte)를 여러 개 만든다. 이 때 이 spte는 물리적 메모리나 스왑에 할당되지 않고 메타데이터만 저장되어 있는 상태이다.

나중에 page fault가 발생하면 page fault handler은 spte의 상태를 토대로 동작을 처리한다. 이것이 `PAGE_LAZY`임을 확인했으면, page fault handler은 `spt_lazy_to_memory()` 함수를 이용해 앞서 저장한 메타데이터를 이용해 정보를 실제 물리 frame에 저장하고 이를 spte에 연결해 준다.

결론적으로, lazy loading은 “`PAGE_LAZY` 상태를 가지는 spte 여러 개 추가”로 단순하게 요약할 수 있다.

2.3. 논의

2.3.1. Design report와 달라진 점

high level에서는 크게 달라진 점이 없다. 조금 더 low level한 측면에서는, **3. Supplemental Page Table**에서도 설명하겠지만, OS의 기능을 구현하는 프로그래머의 입장에서는 frame이나 swap을 신경쓰지 않고 구현의 대부분의 operation을 `spt_xxxxx()` 함수를 이용해서 할 수 있도록 했다. 이것들만 사용해 page의 상태만 신경 쓰며 코딩할 수 있도록 세팅을 해 두었다. 이것이 디자인 레포트를 작성했을 때는 제대로 생각하지 않았던 제일 달라진 점이다.

2.3.2. 배운점

Lazy loading을 이용하면 RAM의 크기보다도 더 큰 파일을 여러 개 여는 것도 가능하다는 점을 새로 배우게 되었다. 단순히 효율의 문제가 아니라 구동 가능 여부의 문제이므로 생각보다 중요하다고 생각했다.

2.3.3. 한계점

이 부분은 함수를 이용한 encapsulation으로 깔끔하게 구현한 것 같아 한계점을 작성하지 않겠다.

3. Supplemental Page Table

3.1. 목표

크게 두 가지 목표가 있다. 첫 번째로 frame와 swap의 구현을 위해 vm의 각 page에 추가적인 메타데이터를 저장하는 것이다(자료구조 측면). 두 번째로는 앞으로 OS의 high level 기능을 구현할 때 (lazy load, stack growth, memory mapping) 프로그래머가 frame이나 swap memory같이 물리적인 측면을 고려하지 않고 기능구현에 집중할 수 있도록 interface를 제공해 주는 것이다 (함수 제공 측면). 이렇게 하면 프로그래머는 spt의 state와 spt의 함수들만을 신경쓰며 로직을 작성할 수 있다. 두 번째 목표는 requirements에 있지는 않지만 다른 기능 구현의 간결성 및 코드 가독성을 높이기 위해 추가하였다.

3.2. 구현 방식

3.2.1. 자료 구조

S-page table은 hash table로 구현했다. Virtual page는 매우 많이 생성될 것으로 예상되며, user virtual address를 이용해 entry를 얻어야 하는 경우가 많은데, 이 탐색을 가장 효율적으로 할 수 있는 자료 구조이기 때문이다. Table의 entry에 해당하는 `struct spt_entry`는 다음과 같이 정의했다.

```
struct spt_entry
{
    void *uaddr;           // user virtual address (key for hash table)
    enum page_status status; // status of the page

    // for PAGE_MEMORY
    struct frame *frame;    // frame the page is loaded into

    // for PAGE_LAZY
    struct file *file;      // file mapped to (if any)
    off_t offset;          // offset in file
    size_t read_bytes;     // bytes to read from file
    size_t zero_bytes;     // bytes to be zeroed

    // for PAGE_SWAP
    size_t swap_index;     // index in swap table

    bool mmap;             // is this page memory-mapped?
    bool writable;         // is page writable?
    struct hash_elem elem; // for S-page hash table
};
```

- 가장 중요한 member은 `page_status status`이다. 이가 가질 수 있는 state는 page의 정보가 물리적으로 저장된 위치와 관련이 깊으며, `PAGE_LAZY`, `PAGE_MEMORY`, `PAGE_SWAP`의 세 가지 상태를 갖는다.
- 그 외의 공용 member로는 `bool mmap`, `bool writable`이 있다. `mmap`의 정보에 따라 evict logic이 달라지고, `writable` 정보도 저장한다.
- Page의 state에 따라 중요해지는 member이 달라진다.
 - `PAGE_LAZY`의 경우 파일로 읽어오기 위해 필요한 메타데이터를 저장한다.
 - `PAGE_MEMORY`의 경우 할당된 frame의 frame table entry를 저장한다. 참고로 우리 구현에서는 `PAGE_MEMORY`가 다시 `PAGE_LAZY`가 되는 경우가 존재할 수 있으므로, file 메타데이터는 지우지 않고 가지고 있어야 한다.
 - `PAGE_SWAP`의 경우 swap table에 저장된 위치만 저장해도 나중에 swap에서 불러오기가 가능하다.

3.2.2. 알고리즘 - **spt.c**에 정의된 함수들

본격적으로 정의한 함수들을 설명하기 전, **PAGE_MEMORY** 상태에 대해 첨언하겠다. 어떤 page가 다른 상태에서 **PAGE_MEMORY**로 바뀌거나, **PAGE_MEMORY**에서 다른 상태로 바뀌는 경우 frame table와의 호응 말고도 process의 page directory와의 호응도 같이 해 주어야 함을 염두에 두어야 한다. **PAGE_MEMORY**가 될 때에는 **palloc_get_page()**와 **pagedir_install_page()**를, **PAGE_MEMORY**에서 벗어날 때는 **palloc_free_page()**와 **pagedir_clear_page()**를 호출해야 한다. 여기서 **palloc**관련 함수는 **frame.c**에 정의된 **frame_alloc()**과 **frame_free()**가 대신 호출해 주고 있다.

아래 두 함수는 새로운 spte를 생성할 때 사용하는 함수들이다.

- **spt_initialize_as_memory()**: page를 **PAGE_MEMORY** 상태로 생성해 주는 함수이다. frame 하나를 할당해 주고, **pagedir_install_page()**를 호출한 다음 spte 엔트리를 만들어 frame table entry와 연결해 준다.
- **spt_initialize_file_as_lazy()**: 파일 하나를 받으면 파일의 모든 정보를 **PAGE_LAZY** 상태의 spte로 만들어 저장해 주는 함수이다. 이 경우 frame이나 swap과는 관련이 없으므로 spte만 잘 만들어 주면 된다.

아래는 spte의 상태를 바꾸기 위해 정의한 함수들이다. 네 함수 모두 **struct spt_entry**만을 input으로 받는다.

- **spt_memory_to_swap()**: spte를 **PAGE_MEMORY**에서 **PAGE_SWAP**으로 바꾸어 주는 함수이다. 우선 **swap_out**을 통해 swap에 정보를 저장하고 swap index를 받은 뒤, 원래 가지고 있던 frame 정보를 해제해 주면 된다. 여기에는 **frame_free()**를 부르는 것에 더해 **pagedir_clear_page()**도 불러 주면 된다.
- **spt_memory_to_lazy()**: spte를 **PAGE_MEMORY**에서 **PAGE_LAZY**로 바꾸는 함수이다. mmap이었으면 dirty bit에 따라서 file로 writeback을 하고, 위에 설명했듯 **frame_free()**와 **pagedir_clear_page()**를 해 주면 된다.
- **spt_swap_to_memory()**: spte를 **PAGE_SWAP**에서 **PAGE_MEMORY**로 바꾸는 함수이다. **frame_alloc()**을 불러 프레임을 할당해준 뒤 **swap_in()**으로 내용물을 불러오고, **pagedir_install_page()**를 통해 물리적으로 할당해 주면 된다. Memory가 가득찼을 때의 eviction algorithm은 앞에서 설명했듯 **frame_alloc()**에서 처리해 준다.
- **spt_lazy_to_memory()**: spte를 **PAGE_LAZY**에서 **PAGE_MEMORY**로 바꾸어 준다. 직전의 함수와 거의 동일한데 **swap_in()** 대신 **load_page()** 함수로 정보를 적어 주면 된다.

3.2.3. 알고리즘 – Page Fault Handler

이제 spte의 state가 제일 중요하게 작용하는 알고리즘 중 하나인 page fault handler의 코드를 살펴 보자.

위에서 고려한 함수들을 이용하면 어렵지 않게 구현이 됨을 확인할 수 있다.

우선 user memory access에 대해 알아야 한다. 우리는 이번에 project 2와 다르게, pointer이 user address 구간에 있는지만 검사한 뒤 구체적인 처리를 page fault handler로 옮겼다. stack growth detection을 여기에서 하기 위한 목적이 제일 크다.

```
static void
page_fault_handler (struct intr_frame *f, void *fault_addr, bool not_present, bool write, bool user)
{
    struct thread *t = thread_current ();
    void *upage = pg_round_down (fault_addr);

    // Rights violation: no lazy load possible.
    if (!not_present)
        process_cleanup (-1);

    void *esp = user ? f->esp : t->esp;
    struct spt_entry *spte = spt_lookup (&t->s_page_table, upage);

    // If no spte, it's either stack growth, or invalid access.
    if (spte == NULL) {
        if (!(PHYS_BASE - MAX_STACK_SIZE) <= fault_addr &&
            fault_addr < PHYS_BASE &&
            esp - 32 <= fault_addr))
            process_cleanup (-1);

        else // Stack growth
        {
            if (!spt_initialize_as_memory (upage))
                process_cleanup (-1);
            // Here, stack creation is successful
            return;
        }
    }
    switch (spte->status)
    {
        // Already mapped but got not-present, so treat as error.
        case PAGE_MEMORY:
            process_cleanup (-1);

        case PAGE_LAZY:
            if (!spt_lazy_to_memory (spte))
                process_cleanup (-1);
            break;

        case PAGE_SWAP:
            if (!spt_swap_to_memory (spte))
                process_cleanup (-1);
            break;

        default:
            process_cleanup (-1);
    }
    return;
}
```

함수의 전부를 불러왔다. page fault의 종류만 분류하면, 그 이후로는 전부 `spt_xxx()` 함수를 통해 처리할 수 있음을 확인할 수 있다. frame이나 swap을 관리하는 것은 함수 내부의 역할이지, 프로그래머가 신경쓸 부분이 아니다.

3.3. 논의

3.3.1. Design report와 달라진 점

뒤의 4. Stack Growth에서도 설명하겠지만, `PAGE_STACK` 상태를 없앴다. 원래의 구현 아이디어에서는 stack으로 allocate될 page를 현재 할당된 stack들의 bottom보다 하나 아래 페이지를 `PAGE_STACK`으로 미리 로딩해놓았는데, 이렇게 하면 문제점이 많아 그 아이디어를 기각했다.

그 외에, high level에서의 s-page table의 구조나 page fault handling logic은 크게 달라지지 않았다.

3.3.2. 배운 점

이부분에서 특히 코드를 가독성있게 짜는 연습을 많이 한 것 같다. 복잡한 부분들을 내부적으로 처리해 함수로 꼭꼭 숨긴 덕분에 다른 코드들의 가독성과 코드의 재사용성을 높일 수 있었다.

3.3.3. 한계

이 부분은 스스로 느끼기에도 꽤나 깔끔하게 짰 것 같아 자랑스럽게 생각하는 부분이다. 따라서 한계점을 작성하지 않겠다.

4. Stack Growth

4.1. 목표

기존 핀토스는 페이지 폴트가 발생하면 무조건 커널 패닉을 일으켰다. 또한 스택은 1 page size로 고정되어 있어, 만약 스택이 꽉 찬 상태에서 추가로 push를 하려고 하는 경우 페이지 폴트가 일어나고, 커널 패닉으로 이어졌었다. 이제 필요에 따라 스택을 페이지 단위로 증가시킬 수 있게 만들어 일정 범위까지는 스택이 자라날 수 있도록 해야 한다.

4.2. 알고리즘

우선 프로세스가 처음 생성될 때, 첫 번째 스택을 만드는 과정을 살펴보자. `load()` 함수가 실행되면, `setup_stack()` 함수가 이어서 호출된다. 이때 `setup_stack()`은 lazy loading 없이 바로 `spt_initialize_as_memory()` 함수를 호출한다. `spt_initialize_as_memory()` 함수는 프레임을 할당하는 역할을 한다.

Page fault가 발생하면 page fault handler가 호출된다. `pagedir/exception.c`의 `page_fault_handler()`는 다음과 같은 순서로 처리한다.

1. page fault가 발생한 주소를 page align 한다(`pg_round_down()`).
2. 해당 페이지의 spt를 보고, spt가 null인지 확인한다. Null이라면 프레임이 할당되지 않은 상태로 틀린 참조이거나 스택이 자라야 하거나 둘 중 하나인 상황이다.
3. 스택이 자라야 하는 조건은 다음과 같다.
 - a. `fault_addr`이 사전에 정의된 user stack 영역 안에 포함되어야 한다.
 - b. `esp - 32 <= fault_addr` 이어야 한다.
4. 스택이 자라야 한다면 `spt_initialize_as_memory(upage)`를 호출한다. 여기서 프레임 할당과 spt setup이 이뤄진다.
5. 만약 `spt_initialize_as_memory()`가 실패했거나, 스택이 자랄 조건이 아니었다면 `process_cleanup(-1)`을 호출해 프로세스를 종료시킨다.

4.3. 논의

4.3.1. Design report와 달라진 점

디자인레포트와 비교해서 가장 많이 달라진 부분이다. 우선 디자인 레포트에서 작성한 스택 구현 계획을 요약하면 다음과 같다.

1. `setup_stack()` 함수에서, 바로 frame에 할당된 최상단 스택 바로 밑에 `PAGE_STACK` 딱 하나를 lazy-load.
2. 이후 page fault 시 포인터가 `PAGE_STACK` spte에 있었으면 stack을 준다.

하지만 이렇게 하면 다음과 같은 문제들이 발생한다.

1. 스택이 크게크게 증가하는 경우 몇 페이지를 스킵하고 페이지 할당을 받을 수 있다. (ex: `sub esp, 0x2000` 이후 `esp`를 dereference하기) 하지만 우리의 첫 구현계획은 이런 경우 page fault에서 recover하지 못한다.
2. 우리 현재의 가장 아래 stack page의 바로 아래 page에 대해 page fault가 나면, 그 dereference가 우리의 `esp`값과 아주 멀어도 (ex: `0x60`바이트만큼) 우리의 구현계획은 이를 stack에 접근했다고 판단했을 것이다. 이 경우에는 반대로 page fault에서 프로세스를 죽여야 한다.

이런 이유들로 `PAGE_STACK`을 이용한 구현은 폐기했으며, “stack 영역에 존재 및 `esp`와 가까움” 정도의 휴리스틱으로 stack에 접근했는지의 여부를 결정하기로 했다.

4.3.2. 배운 점

어떤 문제에 대해 나름의 답을 생각해 보고, 그 생각을 구현해 보고, 처음에 생각했던 답이 결국 효과적이지 못하다는 결론을 내린 후 답을 다시 생각해 보는 과정 자체가 문제해결 능력에 도움이 되었다고 생각한다.

4.3.3. 한계

“`esp - 32 <= fault_addr`”의 휴리스틱은 핀토스 매뉴얼에도 적혀 있지만, 무언가 애매하다는 기분이 자꾸 든다. `pusha` 명령어가 아니면서 `esp - 32`에 있는 address를 요청했을 수도 있는데 이런 요청은 invalid할 가능성이 크다. 그렇다고 해서 page fault가 났던 instruction을 들여다 보는 것은 조금 비쌀 수도 있다는 생각이 든다. 이런저런 방법이 있겠지만 어떤 휴리스틱도 cost와 validity 사이의 밸런스를 모두 잡기는 어렵다는 한계가 있다.

5. File Memory Mapping

5.1. 목표

파일을 메모리에 접근하는 것처럼 사용하는 것을 목표로 한다. 이렇게 하면 별도의 시스템 콜(read, write)을 사용하지 않고 파일의 내용을 수정할 수 있다는 장점이 있다. 또한 크기가 큰 파일도 lazy loading 방식을 활용하기 때문에 메모리를 효율적으로 활용할 수 있게 된다.

5.2. 구현 방식

5.2.1. 자료 구조

```
struct mmap_file {
    mapid_t mapping;
    struct file *file;
    void *addr;
    struct list_elem elem;
};
```

위의 구조체는 `vm/mmap.h`에 위치해 있고 각 필드에 대해 설명하자면 다음과 같다.

- `mapid_t mapping`: 1부터 시작하는 매핑 아이디
- `struct file *file`: re-opened 된 파일 포인터
- `void *addr`: 파일 매핑이 시작되는 가상 메모리 주소
- `struct list_elem elem`: 리스트의 원소로 사용하기 위함

이때 매핑 아이디로는 전역 변수 `next_id`를 선언해 사용했다.

또한 위에서 살펴봤던 `spt_entry`의 필드인 `bool mmap`도 여기서 활용된다.

```
...
bool mmap;           // is this page memory-mapped?
...
```

이는 프레임이 evict 될 때, 변경 사항이 있을 경우 내용을 파일에 write back 해야 할지 아니면 swap으로 이동해야 할지를 결정하기 위해 필요하다.

마지막으로 스레드 구조체에 추가된 필드를 살펴보자.

```
#ifndef VM
...
struct list mmap_list;      /* List of memory-mapped files */
#endif
```

이는 각 스레드마다 자신만의 memory map list를 갖고 있어야 하기 때문에 스레드 구조체에 선언한 것이다.

5.2.2. 알고리즘

먼저 `sys_mmap()` 함수를 보면 알고리즘은 다음과 같다.

1. 주어진 `fd`와 매핑 될 메모리 포인터가 valid 한지 검사
 - a. `fd < 2`, `addr==NULL`, `pg_ofs(addr)!=0` 인지
 - b. `file`이 `NULL`이거나 길이가 0인지
 - c. 파일을 매핑하려고 하는 영역에 이미 다른 페이지가 할당 되어 있는 경우 -1 반환

2. 주어진 `fd`에 해당하는 file을 다시 열어서 기존 file이 close 되더라도 매핑이 이어지도록 한다.
3. 새로운 `mmap_file` 포인터를 만들고 mapping, file, addr을 올바르게 입력해준다.
4. 마지막으로 `spt_initialize_file_as_lazy()`를 호출해 페이지를 lazy loading 해준다.
만약 실패한다면 리스트 원소를 삭제하고 열었던 파일도 다시 닫는다.

다음으로 `sys_munmap()` 함수의 알고리즘을 보자. 이 함수는 먼저 mapping을 이용해 mmap원소를 찾고, 해당하는 mmap이 존재할 경우 `mmap_unmap_and_flush()` 함수를 호출한다. 이 함수는 `mmap.c`에 정의되어 있다.

1. 유효한 매개변수인지 확인, 파일 길이 획득
2. 하나의 매핑에 여러 페이지가 매핑 되어 있을 수도 있으므로(lazy loading) 페이지마다 순회하며 spt entry를 획득한다.
3. 만약 spte의 status가 `PAGE_MEMORY` 라면 dirty bit을 확인해 write back을 필요한 경우 수행한다.
4. 이후 해당 spte가 가리키던 프레임을 해제하고 해당 spte를 `s_page_table`에서 삭제한다.
5. 반복문이 종료되면(모든 페이지에 대해 순회 한 이후라면) mmap이 가리키던 file을 닫고, `mmap_list`에서 해당 mmap을 지운 다음 free 시킨다.

5.3. 논의

5.3.1. Design report와 달라진 점

디테일한 구현에 있어서 달라진 부분은 있지만, 큰 틀에서 디자인 레포트와 달라진 부분은 없다고 생각한다.

5.3.2. 배운 점

매번 시스템 콜을 할 필요 없이 파일을 읽고 수정하는 memory mapped file 이라는 개념을 이해할 수 있었다. 또 mmap 덕분에 frame eviction logic에서 고려해야 되는 케이스가 늘어났는데, 이 과정에서 frame eviction에 대한 이해도 더 할 수 있었다.

5.3.3. 한계

생각보다 잘 구현한 것 같다. lazy loading도 `spt.c`에서 정의했던 함수를 이용해서 잘 구현한 것 같다. (`spt_initialize_file_as_lazy()`) 한 가지 아쉬운 점은 해당 함수로 보내는 argument가 너무 많다는 것이다. 이를 조금 더 간소화해서 `s_page_table`, `file`, `writable`, `mark_mmap` 정도만 보낸 뒤 나머지 argument는 `spt_initialize_file_as_lazy()` 함수 내부에서 구현하는 것도 가능했을 것이라고 생각된다. (이게 더 깔끔한 코드일 것 같기도 하다.) 하지만 함수를 이렇게 디자인한 이유는 `userprog/process.c`에서의 `load()` 함수를 project 2의 코드와 공유하기 위해서였다. 아래에 그 코드를 붙여넣었다.

```
#ifdef VM
    if (!spt_initialize_file_as_lazy (&t->s_page_table, file, file_page, (uint8_t *) mem_page, read_bytes,
    zero_bytes, writable, false)) goto done;
#else
    if (!load_segment (file, file_page, (void *) mem_page, read_bytes, zero_bytes, writable)) goto done;
```

6. Swap Table

6.1. 목표

프로세스가 여러개 실행 중인 상황에서, 메모리는 쉽게 부족해질 수 있다. 이때 메모리 공간이 추가적으로 필요한 경우 프로세스를 강제 종료 시키기 전에 swap disk를 이용할 수 있다. 이는 메모리에 저장된 내용을 일시적으로 디스크에 보관하도록 하는 것으로, swap table이 관리하게 된다.

6.2. 구현 방식

6.2.1. 자료 구조

Swap table은 비트맵(bit-map)이라는 자료구조로 관리하게 된다. 이는 `swap.c`에 전역변수로 선언 되어 있고, `src/lib/kernel/bitmap.c`에 정의된 bitmap이라는 구조체를 사용한다. 이외에도 자료구조는 코드에서 다음과 같이 정의된다.

```
static struct block *swap_device;
static size_t swap_total_pages;
static struct bitmap *swap_bitmap;
static struct lock swap_lock;
```

- `swap_device`: swap disk를 참조하는 block 구조체 타입의 포인터이다. `block_read()`, `block_write()` 등을 사용할 때 이용한다.
- `swap_total_pages`: `swap_device`에 저장할 수 있는 총 페이지 수이다.
- `swap_bitmap`: Swap table 역할을 하는 비트맵이다.
- `swap_lock`: swap table에 접근하는 것을 관리하는 동기화 도구이다.

6.3.2. 알고리즘

`swap.c`에 있는 함수를 하나씩 설명하겠다. 먼저 `swap_init()`의 알고리즘을 살펴보자.

1. `block_get_role(BLOCK_SWAP)`을 호출해 `swap_device`를 초기화 한다. 실패할 경우 패닉을 일으킨다.
2. `swap_total_pages`를 계산하고 그 크기만큼 비트맵을 만들어 `swap_bitmap`을 초기화한다.
3. `swap_lock`을 초기화한다.

다음으로 살펴볼 함수는 `swap_in()`으로, swap disk에 저장되어 있던 데이터를 메모리로 다시 읽어오는 역할을 한다.

1. `swap_lock`을 acquire 한다.
2. 총 8번(Page 크기 / block sector 크기) 반복을 하며 `block_read()`를 수행한다.
3. 이제 해당 index의 `swap_bitmap`을 false로 표시해 swap에 이용할 수 있는 영역이 되었음을 기록한다.
4. `swap_lock`을 release 한다.

`swap_free()` 함수도 있는데, 이는 `swap_in()`에서 read를 하지 않는 버전의 함수이다. (즉, `swap_bitmap` 정보만 false로 바꿈) 이는 뒤의 **7. On Process Termination** 부분에서 사용하게 된다.

마지막으로 `swap_out()`은 메모리에서 swap disk 영역으로 내용을 저장할 때 호출되는 함수이다.

1. `swap_lock`을 acquire 한다.
2. `bitmap_scan_and_flip()`을 호출해 사용 가능한 swap 영역을 가리키는 `swap_index`를 찾는다.
3. 마찬가지로 8번 반복하며 `block_write()`을 수행해 페이지에 저장된 내용을 swap 영역에 옮긴다.
4. `swap_lock`을 release 한다.

6.3. 논의

6.3.1. Design report와 달라진 점

디자인 레포트에서 계획했던 대로 구현한 것 같다. 한 가지 사소한 점은, 디자인 레포트에서는 `block_read()`을 스스로 계산해서 8번 불러올 것이라고 계산했지만, 구현에서는 잘 정의된 매크로들을 이용해 구현했다. 이렇게 하면 코드의 재사용성이 좋아진다.

6.3.2. 배운 점

컴퓨터를 사용하면서 스왑 메모리나 스왑 디스크 같은 개념들을 자주 접했는데 이번에 직접 구현을 하며 간략하게나마 어떻게 작동하는 개념인지를 알 수 있었다. 비트맵이라고 하는 자료구조도 핀토스에서는 처음 사용해보았다.

6.3.3. 한계

구조체 자체는 간단해서 생겼을만한 한계가 잘 떠오르지 않는다.

7. On Process Termination

7.1. 목표

User process이 종료될 때, process과 관련되어 할당되었던 모든 memory를 deallocate하는 기능을 구현해야 한다. Deallocation을 제대로 수행하지 않으면 memory leak이 발생하여, 아무도 사용하지 않는데도 활용할 수 없는 memory가 늘어나서 os의 성능이 엄청나게 저하된다.

7.2. 알고리즘

Process가 종료되는 상황에서는 모두 `process_cleanup()` 이라는 함수를 호출하도록 설계했다.

따라서 `process_cleanup()` 함수만 제대로 실행되면 프로세스의 종료를 올바르게 처리했다고 볼 수 있다. 기존에 프로세스 종료에 사용하던 로직은 위에 두고, `#ifdef VM`으로 감싸진 부분만 보면 된다.

```
#ifdef VM
// Flush all mmap'd regions on exit, even if user didn't call munmap.
// Do this before destroying the SPT and page directory.
for (struct list_elem *e = list_begin(&cur->mmap_list); e !=
list_end(&cur->mmap_list); )
{
    struct mmap_file *m = list_entry (e, struct mmap_file, elem);
    e = list_next(e);

    mmap_unmap_and_flush (cur, m);
}

// Now destroy the supplemental page table
hash_destroy (&cur->s_page_table, spt_destroy_entry);
#endif
```

먼저 해당 프로세스의 `mmap_list`를 순회하면서 `mmap_unmap_and_flush()` 함수를 호출해준다. 이 함수는 5.2.2에서 알고리즘을 설명한 바 있다.

mmap의 메모리 해제가 종료되면 s-page table을 해제해준다. 이는 `hash_destroy()` 함수로 수행한다.

`hash_destroy()`의 destructor로 사용되는 `spt_destroy_entry()`를 사용하는데,

`hash_destroy()`는 이 destructor를 hash의 모든 element에 대해 수행한 뒤, hash table 자체를 부수고 종료하는 함수이다.

`spt_destroy_entry()`는 spt entry를 찾고, 해당 entry의 status에 따라 다른 처리를 한다.

- 만약 status가 `PAGE_MEMORY` 라면 해당 프레임을 갖고 있는 스레드의 페이지 디렉토리에서 자신을 없앤다. 이후 `frame_free()`를 호출해 해당 frame을 해제한다.
- 만약 status가 `PAGE_SWAP` 이라면 `swap_free()`를 호출해 해당 entry의 swap_index에 해당하는 bitmap index를 false로 바꿔준다.

마지막으로 page table도 해제 해주어야 하는데, 이는 `process_exit()`의 밑 부분에 이미 작성되어 있어서 별도로 처리할 내용은 없었다.

7.3. 논의

7.3.1. Design report 와 달라진 점

원래는 `process_exit()`에 내용을 추가하려고 했는데, 아예 새로운 함수인 `process_cleanup()`이라는 함수를 만들고 그 안에 로직을 추가하는 방식으로 해결했다.

7.3.2. 배운 점

프로젝트2를 구현할 때 자식 프로세스는 부모 프로세스가 종료되기 전까지 semaphore를 이용해 기다리도록 설계했었다. 이런 방식으로 orphan process를 처리했었는데, 자식 프로세스의 frame table은 해제가 되지만 `palloc_free_page()`는 실행되지 않아 결과적으로 실질적인 메모리 해제가 이뤄지지 않는 문제가 발생했었다. 이는 `palloc_free_page()`을 `frame_free()`에 명시적으로 호출함으로써 해결했다. 메모리 누수 방지가 설계할 때 얼마나 헛갈리는 부분인지를 배울 수 있었다.

7.3.3. 한계점

테스트 케이스는 다 통과되도록 메모리 누수를 처리했지만 더 꼼꼼하게 확인해보면 메모리 누수가 있을 수도 있겠다는 불안감이 있다.